



HES-SO Master

Master in Engineering - Computer Science

Mini projet intégré

CSEL - Construction de Systèmes Embarqués sur Linux

Auteurs

Carolina Inácio Pereira, Gaby Roch, Simon Mirkovitch

Dépot Git

<https://github.com/g-roch/mse-csel-workspace>

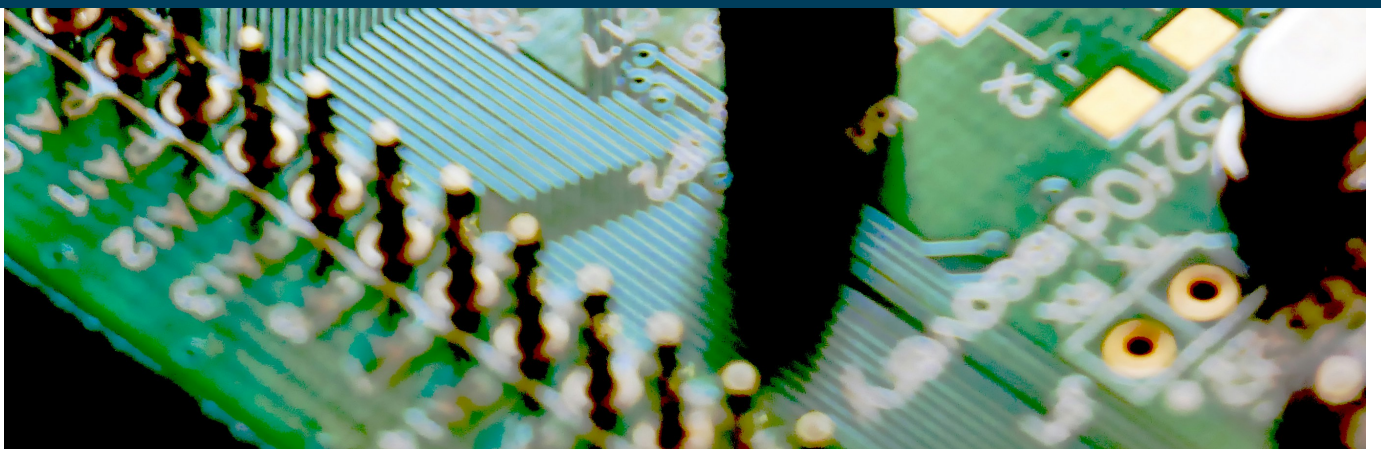


Table des matières

1	Introduction	2
2	Résumé du mini-projet	3
3	Ce qui a été appris / exercé	4
4	Remarques / À retenir	5
4.1	Multiplexage des entrées des boutons	5
4.2	Mapping des boutons avec les actions	6
4.3	Modification du <i>Device Tree</i>	6
4.4	Gestion des conflits	8
5	Conclusion	9
6	Annexes	10
A.	Multiplexage des boutons avec <code>select</code> et <code>readfds</code>	10
B.	Structure du dépôt	12

1 Introduction

Ce rapport présente les travaux réalisés dans le cadre du mini-projet de construction de systèmes embarqués sous Linux. L'objectif était de concevoir une application complète de gestion de la vitesse de rotation d'un ventilateur (ici remplacé par une LED qui clignote) en fonction de la température du processeur, avec modes automatique et manuel.

Le système repose sur deux composants principaux : un module noyau supervisant la température du CPU et pilotant une LED via le *sysfs* et un *daemon* en espace utilisateur gérant l'interface physique (boutons, LED, écran OLED). L'interface IPC avec application en ligne de commande, étant proposée en option dans la consigne (à la place de la gestion des boutons), n'a pas été implémentée.

Ce projet mobilise l'ensemble des compétences développées au fil des laboratoires : développement de modules noyaux, gestion des GPIOs et des *timers*, communication inter-processus, ainsi que la mise en œuvre d'un système embarqué complet sur NanoPi Neo Plus 2 sous *Buildroot*.

L'objectif de ce rapport est de résumer les étapes de réalisation, de documenter les choix d'implémentation et de partager les remarques et retours d'expérience associés à ce mini-projet.

Le code source est disponible dans le dépôt du groupe (voir Annexe B.).

2 Résumé du mini-projet

Le mini-projet consiste à simuler la gestion d'un ventilateur piloté par la température du CPU d'un NanoPi Neo Plus 2. Un module noyau lit périodiquement la température via `linux/thermal.h` et ajuste la fréquence de clignotement d'une LED Status (entre 2 et 20 Hz) selon quatre seuils :

- Température $< 35^{\circ}C$ $\rightarrow$ fréquence de 2Hz
- Température $< 40^{\circ}C$ $\rightarrow$ fréquence de 5Hz
- Température $< 45^{\circ}C$ $\rightarrow$ fréquence de 10Hz
- Température $\geq 45^{\circ}C$ $\rightarrow$ fréquence de 20Hz

Il expose via le `sysfs` une interface permettant de sélectionner le mode de fonctionnement et de régler manuellement la fréquence.

En parallèle, un *daemon* en espace utilisateur gère l'interface physique : les boutons `k1/` `k2` permettent d'ajuster la fréquence, `k3` bascule entre mode automatique et manuel, et la LED Power signale les appuis. L'écran OLED affiche en temps réel le mode actif, la température et la fréquence courante.

3 Ce qui a été appris / exercé

Le tableau ci-dessous présente les compétences exercées durant les laboratoires selon trois niveaux d'acquisition : *non acquis*, *acquis, mais à exercer* et *parfaitement acquis*.

Point / Sujet	Carolina	Gaby	Simon
Mise en œuvre d'un système embarqué sous Linux	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Développement de modules noyaux	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Développement de pilotes de périphériques	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Traitement des fichiers ordinaires et spéciaux	Acquis, mais à exercer	Parfaitement acquis	Acquis, mais à exercer
Développement d'applications multitâches	Acquis, mais à exercer	Parfaitement acquis	Acquis, mais à exercer
Gestion des ressources du CPU	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Optimisation	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer

4 Remarques / À retenir

4.1 Multiplexage des entrées des boutons

Pour surveiller les trois boutons simultanément, une première approche avec `select` et `readfds` a été testée. Le code dans l'Annexe A. illustre le prototype initial.

Cette implémentation produisait une consommation CPU de 100% et générerait des événements en continu, même sans changement d'état des boutons, ce qui imposait de gérer manuellement l'état précédent pour détecter un vrai appui.

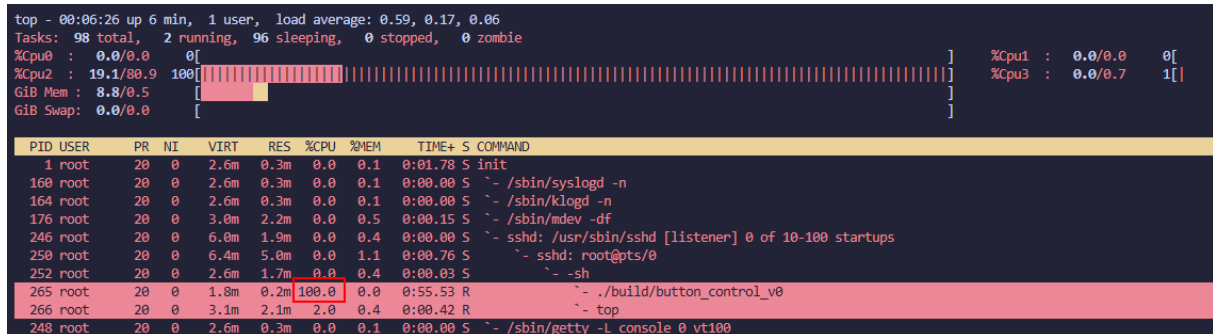


Fig. 1. – Multiplexage des boutons GPIO avec `select` et `readfds` utilise 100% du CPU

La pression d'un bouton, par exemple k2, donnait le résultat suivant à la console :

```
K2 pressed
K2 pressed
K2 pressed
K2 pressed
K2 pressed
K2 pressed
[...]
```

La cause n'était pas immédiatement évidente : en croyant que `select` était simplement trop limité pour ce cas, l'implémentation a été migrée vers `epoll`, l'autre mécanisme vu en cours. Avec `epoll` et l'événement `EPOLLPRI`, le résultat était nettement plus simple : plus de gestion d'état, de spam d'événements ou de consommation exagérée du CPU.

La vraie cause a été comprise par la suite : les fichiers `/sys/class/gpio/gpioX/value` sont des fichiers spéciaux du noyau qui sont « toujours prêts à être lus ». Ces fichiers retournent donc instantanément la valeur courante de la broche, ce qui fait que `select` avec `readfds` ne bloque jamais, créant une boucle occupée (*busy waiting*).

La documentation noyau précise que pour n'être notifié qu'aux changements de front (*edge transitions*), il faut placer les descripteurs dans `exceptfds` avec `select`, ou utiliser `POLLPRI/EPOLLPRI` avec `poll/epoll`¹ :

«If you use `poll(2)`, set the events `POLLPRI` and `POLLERR`. If you use `select(2)`, set the file descriptor in `exceptfds`.»

¹<https://www.kernel.org/doc/Documentation/admin-guide/gpio/sysfs.rst>

Ces deux approches sont sémantiquement équivalentes : toutes les deux surveillent le canal des événements prioritaires, que le noyau utilise pour signaler les changements de front GPIO via son mécanisme d'interruption. `epoll` avec `EPOLLPRI` a été conservé puisqu'il était déjà fonctionnel et reste plus adapté que `select` pour surveiller plusieurs descripteurs, les `fd_set` devant être reconstruits à chaque appel avec `select`².

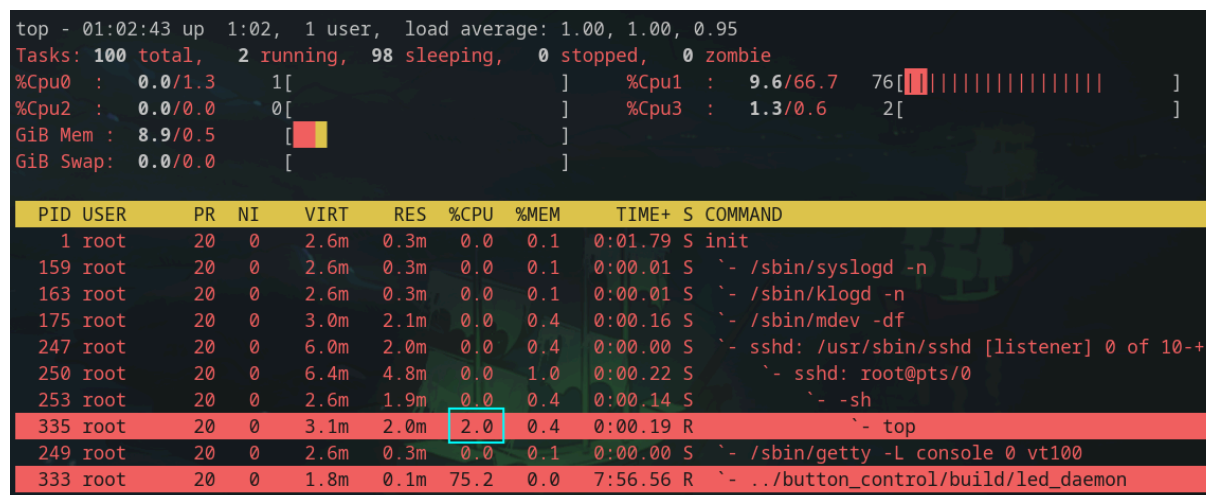


Fig. 2. – Multiplexage des boutons GPIO avec `epoll` et `EPOLLPRI` utilise moins de CPU

4.2 Mapping des boutons avec les actions

La consigne du projet définit S1 comme le bouton d'augmentation de fréquence et S2 comme celui de diminution. Or, sur la carte d'extension, les boutons physiquement positionnés de gauche à droite sont k1 (S1), k2 (S2) et k3 (S3). Cette disposition aurait conduit à une interface contre-intuitive, où l'on augmente la vitesse avec le bouton de gauche et on la diminue avec celui de droite, nous l'avons donc inversé :

- S1 pour diminuer la fréquence
- S2 pour l'augmenter
- S3 reste inchangé : changement de mode

Le comportement fonctionnel reste conforme à la spécification (signalisation sur la LED Power, gestion des niveaux de fréquence), seule l'association bouton/action a été adaptée.

4.3 Modification du *Device Tree*

Afin de pouvoir contrôler l'écran OLED du Nano Pi Neo Plus 2, il est nécessaire d'activer l'interface `i2c` en modifiant la configuration du *Device Tree* :

```
nano board/friendlyarm/nanopi-neo-plus2/nanopi-neo-plus2.dts
```

Shell

²https://man7.org/linux/man-pages/man2/epoll_ctl.2.html

```
GNU nano 7.2
/dts-v1/;

#include "allwinner/sun50i-h5-nanopi-neo-plus2.dts"

/ {
    /delete-node/ leds;
};

&i2c0 {
    status = "okay";
};
```

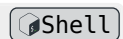
Fig. 3. – Modification du fichier DTS

Cette portion de configuration a été ajoutée :

```
&i2c0 {
    status = "okay";
};
```

Ensuite, reconstruire le noyau avec :

```
make linux-rebuild
make
```



Le fichier nanopi-neo-plus2.dtb correspondant se situe dans /buildroot/output/images :

```
root → /buildroot (csel) $ ls -la output/images/
total 718740
drwxr-xr-x 5 root root    4096 Jun  5 20:01 .
drwxr-xr-x 6 root root    4096 Mar 28 14:20 ..
-rwxr-xr-x 1 root root   20484 Mar 28 14:16 bl31.bin
-rwxr-xr-x 1 root root    269 Mar 28 14:16 boot.scr
-rw-r--r-- 1 root root  67108864 Mar 28 14:35 boot.vfat
drwxr-xr-x 2 root root    4096 Mar 28 14:03 brcm
drwxr-xr-x 2 root root    4096 Mar 28 14:03 cypress
drwxr-xr-x 2 root root    4096 Mar 28 14:35 extlinux
-rw-r--r-- 1 root root  34650624 Jun  5 20:01 Image
-rw-r--r-- 1 root root  11976341 Mar 28 14:20 Image.gz
-rwxr-xr-x 1 root root    28015 Jun  5 20:01 nanopi-neo-plus2.dtb
-rw-r--r-- 1 root root 2147483648 Mar 28 14:35 rootfs.ext2
lrwxrwxrwx 1 root root     11 Mar 28 14:35 rootfs.ext4 -> rootfs.ext2
-rw-r--r-- 1 root root  166942720 Mar 28 14:35 rootfs.tar
-rw-r--r-- 1 root root 2215682048 Mar 28 14:35 sdcard.img
-rwxr-xr-x 1 root root    28247 Jun  5 20:01 sun50i-h5-nanopi-neo-plus2.dtb
-rw-r--r-- 1 root root    32768 Mar 28 14:17 sunxi-spl.bin
-rw-r--r-- 1 root root   610199 Mar 28 14:17 u-boot.bin
-rw-r--r-- 1 root root   631608 Mar 28 14:17 u-boot.itb
```

Fig. 4. – Fichier DTB généré

Enfin, exécuter sync-images.sh pour synchroniser les fichiers du DevContainer avec le système de fichiers local :

```
root → /buildroot (csel) $ sync-images.sh
sending incremental file list
created directory /workspace/buildroot-images
./
[...]
```



```
nanopi-neo-plus2.dtb
      28,015 100%    2.97MB/s    0:00:00 (xfr#6, to-chk=65/72)
[...]
```

Au prochain démarrage, `i2c` est disponible dans `/dev` :

```
/# ls /dev/i*
/dev/i2c-0
```



4.4 Gestion des conflits

Le *daemon* et le module noyau communiquent via trois attributs *sysfs* exposés sous `/sys/kernel/led/`. Pour éviter toute condition de course (*race condition*) sans recourir à des verrous, le principe retenu est celui d'un écrivain unique par attribut : chaque fichier n'a qu'un seul producteur et un seul consommateur, ce qui rend le flux de données naturellement unidirectionnel.

Attribut	Écrivain	Lecteur	Contenu
set_freq	Daemon	Module	Fréquence demandée en mode manuel
auto_freq	Daemon	Module	Mode courant : 0 manuel, 1 automatique
get_freq	Module	Daemon	Fréquence effectivement appliquée à la LED

Tableau 1. – Répartition des rôles : un seul écrivain par attribut

`get_freq` constitue l'unique source de vérité : le module appelle `sysfs_notify()` à chaque changement de fréquence, ce qui réveille immédiatement le daemon via `EPOLLPRI` sans polling actif.

À l'intérieur du module, le partage de la fréquence entre les deux *kthreads* (`blink_thread` et `control_thread`) est géré par une variable atomique (`atomic_t current_freq`) et une file d'attente (`wait_event_interruptible`), évitant tout accès concurrent en mémoire.

5 Conclusion

Ce mini-projet a permis de mettre en pratique de manière concrète et intégrée les compétences acquises au fil des laboratoires, en développant un système embarqué complet sur NanoPi Neo Plus 2. L'articulation entre un module noyau et un *daemon* en espace utilisateur a illustré la séparation des responsabilités propre à l'architecture Linux : le noyau gère les ressources matérielles et expose des interfaces standardisées via le *sysfs*, tandis que l'espace utilisateur orchestre la logique applicative et l'interaction avec l'utilisateur.

Sur le plan technique, les principaux défis rencontrés ont porté sur la gestion des GPIOs par événements avec *epoll*, l'activation de l'interface *i2c* via le *Device Tree* pour l'écran OLED, ainsi que la coordination entre le module noyau et le *daemon* pour maintenir un état cohérent du système.

L'interface IPC, initialement prévue pour offrir un pilotage en ligne de commande, n'a pas pu être réalisée dans le temps imparti. Elle aurait constitué un complément naturel au système, en permettant notamment une intégration dans des scénarios d'automatisation ou de supervision à distance.

Ce projet a été une opportunité de comprendre par la pratique la complexité des systèmes embarqués sous Linux, où les couches matérielle, noyau et espace utilisateur doivent être maîtrisées et coordonnées pour obtenir un comportement fiable et cohérent.

6 Annexes

A. Multiplexage des boutons avec select et readfds

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include <sys/select.h>

#define EXPORT_GPIO_PATH   "/sys/class/gpio/export"
#define UNEXPORT_GPIO_PATH "/sys/class/gpio/unexport"

const int K1_BUTTON_GPIO = 0;
const int K2_BUTTON_GPIO = 2;
const int K3_BUTTON_GPIO = 3;

static int setup_gpio(int gpio, const char *direction, const char *edge)
{
    char path[64];
    FILE *file;

    snprintf(path, sizeof(path), UNEXPORT_GPIO_PATH);
    file = fopen(path, "w");
    if (file) { fprintf(file, "%d", gpio); fclose(file); }

    snprintf(path, sizeof(path), EXPORT_GPIO_PATH);
    file = fopen(path, "w");
    if (file) { fprintf(file, "%d", gpio); fclose(file); }

    snprintf(path, sizeof(path), "/sys/class/gpio/gpio%d/direction", gpio);
    file = fopen(path, "w");
    if (file) { fprintf(file, "%s", direction); fclose(file); }

    if (edge) {
        snprintf(path, sizeof(path), "/sys/class/gpio/gpio%d/edge", gpio);
        file = fopen(path, "w");
        if (file) { fprintf(file, "%s", edge); fclose(file); }
    }

    snprintf(path, sizeof(path), "/sys/class/gpio/gpio%d/value", gpio);
    int fd = open(path, O_RDONLY);
    if (fd == -1) { perror("open gpio value"); return -1; }
    return fd;
}
```

```

int main(void)
{
    int k1_fd = setup_gpio(K1_BUTTON_GPIO, "in", "both");
    int k2_fd = setup_gpio(K2_BUTTON_GPIO, "in", "both");
    int k3_fd = setup_gpio(K3_BUTTON_GPIO, "in", "both");

    int maxfd = (k1_fd > k2_fd ? (k1_fd > k3_fd ? k1_fd : k3_fd) : (k2_fd > k3_fd ?
    k2_fd : k3_fd));
    char value[2];

    while (1) {
        fd_set fd_in;
        FD_ZERO(&fd_in);
        FD_SET(k1_fd, &fd_in);
        FD_SET(k2_fd, &fd_in);
        FD_SET(k3_fd, &fd_in);

        int ret = select(maxfd + 1, &fd_in, NULL, NULL, NULL);
        if (ret == -1) { perror("select"); break; }

        if (FD_ISSET(k1_fd, &fd_in)) {
            lseek(k1_fd, 0, SEEK_SET);
            read(k1_fd, value, sizeof(value));
            if (value[0] == '1') printf("K1 pressed\n");
        }
        if (FD_ISSET(k2_fd, &fd_in)) {
            lseek(k2_fd, 0, SEEK_SET);
            read(k2_fd, value, sizeof(value));
            if (value[0] == '1') printf("K2 pressed\n");
        }
        if (FD_ISSET(k3_fd, &fd_in)) {
            lseek(k3_fd, 0, SEEK_SET);
            read(k3_fd, value, sizeof(value));
            if (value[0] == '1') printf("K3 pressed\n");
        }
    }

    return 0;
}

```

B. Structure du dépôt

Le code source complet est disponible sur le dépôt GitHub du groupe (lien en page de garde). Les composants développés se trouvent aux emplacements suivants :

Composant	Chemin
Module noyau	src/07_miniproj/module/
Daemon	src/07_miniproj/button_control/

La bibliothèque OLED utilisée (ssd1306) est celle fournie par l'enseignant ; les fichiers ont été copiés dans le dossier du *daemon* sans modification.